TECHNICAL DEEP-DIVE REPORT Resolved

CDN Query Parameter Normalization Bug

Detailed Infrastructure, Router, and API Breakdown (Localhost vs. Production)

Prepared by: Junior Developer (Frontend & Systems)

Date: May 27, 2026

Approved by: Lead Systems Architect

Status: Client-Side Safeguard Deployed

1. Executive Summary

During testing of the Ad Genius Concepts V2 dashboard, we discovered a significant behavioral gap between our local development environment (`localhost`) and the production environment (hosted on Vercel/Edge CDN) concerning the acquisition channel filter.

> **The Bug Behavior**
>
> When a user deselected all channels (Meta, TikTok, YouTube, etc.) to view "None" (resulting in an empty filter set), **localhost** correctly showed an empty state (0 records). However, on

> **production**, the interface displayed all records or stale data for all channels, completely ignoring the "None" state.

The issue was traced to Vercel/Edge CDN's automatic query parameter normalization, which stripped empty query strings (e.g. `&channels=`) before passing the request to the Next.js API server. The API route interpreted this missing parameter as "no filter applied" and default-queried all records.

Resolution: We deployed client-side short-circuit guards within the data fetching hooks (`useCachedData.ts` and `useComparisonConcepts.ts`). If the selected channels array is empty, the hooks bypass the network fetch entirely and immediately return empty records. This guarantees consistent behavior across environments and prevents redundant, costly server/database calls.

2. Infrastructure Architecture & Request Lifecycle

To understand why this bug occurred, we must trace the request lifecycle and query parsing behaviors across the development and production stacks.

2.1 The Localhost Environment (Next.js Node.js Server)

In our local development environment, the application runs on a Node.js process using Next.js's built-in development server.

- **Query Parsing:** When the client sends an HTTP request, the Next.js router parses the query string using the V8 engine's standard `URL` interface and `URLSearchParams`.
- **Parameter Preservation:** According to the standard W3C URL specification, query parameters with empty values (e.g., `?channels=`) are valid and are parsed as having a value of `""` (empty string).
- **Direct Route:** Because there is no intermediary reverse proxy or CDN layer on localhost, the request goes directly from the local browser to the Next.js dev server. The query parameters are received exactly as they were written.

2.2 The Production Environment (Vercel Edge Routing & CDN)

In production, the application is deployed on Vercel's cloud platform. The infrastructure introduces several layers between the user's browser and the Next.js serverless functions:

1. **Global Edge Network (Anycast IP):** Directs the user's request to the nearest Vercel Edge

Server.

2. **Vercel Smart Routing & Edge Middleware:** Runs routing logic, header injection, and runs middleware functions.
3. **CDN Caching Layer:** Manages static assets and caches serverless API responses.
4. **Serverless/Edge Functions (AWS Lambda/Vercel Edge):** Executes the Next.js server code.

The Role of Cache Key Normalization

CDNs optimize latency by caching response payloads. To do this, they generate a **Cache Key** (a unique hash) based on the request URL. To prevent "cache pollution" (where slightly different URLs pointing to identical content result in multiple cache misses and duplicate storage), CDNs perform **URL Normalization** by sorting query parameters and **stripping empty query parameters**.

When a request passes through Vercel's Edge CDN, the CDN router determines that `&channels=` is an empty parameter and strips it completely to normalize the cache key:

- **Original client request:** `/api/ad-performance/cached?from=...&sortBy=totalSpend&channels=`
- **Normalized request (forwarded to API):** `/api/ad-performance/cached?from=...&sortBy=totalSpend`

3. Server-Side API Handler Flow (The Vulnerability)

The backend API endpoint resides at `src/app/api/ad-performance/cached/route.ts`. The server-side vulnerability was located in how it extracted and parsed the `channels` parameter from the incoming URL.

3.1 Code-Level Parameter Extraction

The API route extracts query parameters:

```
const channelsParam = url.searchParams.get('channels');
```

According to W3C specifications for `URLSearchParams.get()`:

- If `channels=` is present: `url.searchParams.get('channels')` returns `""` (empty string).
- If `channels` is missing: `url.searchParams.get('channels')` returns `null`.

The API route then parsed this parameter as follows:

```
let selectedChannels: string[] | null = null;
```

```
if (channelsParam !== null) {

 if (channelsParam === '') {

   // Empty string means "no channels selected" (deselect all)

   selectedChannels = [];

 } else {

   // Pass comma-separated list of selected channels

   selectedChannels = channelsParam.split(',').map(c => c.trim()).filter(c => c.length > 0);

 }

}
```

3.2 The Logic Breakdown

- **Localhost Flow:** Since the dev server receives `channels=`, `channelsParam` was `""`. The inner check `channelsParam === ''` evaluated to true, and `selectedChannels` was set to `[]`. The database query filtered for channels in the empty list `[]`, returning 0 records.
- **Production Flow:** Since Vercel Edge CDN stripped `channels=`, `channelsParam` was parsed as `null`. The block `if (channelsParam !== null)` was skipped entirely. `selectedChannels` remained `null`, which the backend interpreted as "no channel filter applied," querying and returning data for all active channels.

4. Evaluated Solutions & Impact Analysis

To resolve this discrepancy, we evaluated three separate engineering approaches: Client-Side Short-Circuiting, Server-Side Parameter Contract Modification, and CDN Caching/Routing Reconfiguration.

Strategy Comparison

Strategy	Implementation Area	Complexity	Database Cost Impact	Regression Risk	Verdict
A: Client-Side Short-Circuit	React Hooks (useCachedData, useComparison)	Low (Pure Client)	**100% reduction** for "None" queries (No network calls)	Low (Client state encapsulated)	**Implemented**
B: Server-Side Contract Rewrite	API Route (route.ts) & Prisma Models	Medium (Client & Server)	Neutral	Medium (Breaks existing API specifications)	Rejected
C: CDN Configuration	vercel.json rewrite / cache config	High (Infrastructure)	Neutral	High (Cache hit rate degradation)	Rejected

4.1 Strategy A: Client-Side Short-Circuit (The Chosen Solution)

This approach intercepts the empty filter array state `[]` inside the fetching hooks. Instead of compiling a query string and firing a fetch request, the hooks immediately update the component state to an empty list, reset page/pagination information, and return.

- **Advantages:** Bypasses BigQuery and PostgreSQL queries entirely for empty states. Since BigQuery queries are billed by the amount of data scanned, preventing these useless queries directly saves cloud operational costs. Also, the transition to the "None" state is instantaneous (0ms) from a UX perspective.
- **Risks:** Must ensure that clearing the client state doesn't leave loading spinner flags active or break pagination state. This has been fully verified and handles all cleanups correctly.

4.2 Strategy B: Server-Side Parameter Contract Modification (Alternative)

This strategy would require changing how the client communicates "None" to the server. Rather than passing an empty parameter (`&channels=`), the client would pass an explicit token indicating a zero-selection filter, such as `&channels=NONE_SELECTED` or a separate query flag `&filterMode=none`.

- **Why It Was Rejected:** It requires changing the API contract on both the client and the server, introducing regression risks if other services call this API. Furthermore, the server would still need to process the request, spin up a serverless instance, authenticate, and respond, wasting cloud resources.

4.3 Strategy C: CDN Caching Configuration (Alternative)

This strategy involves customizing Vercel's caching behaviors (via rewrite settings or header configurations) to prevent stripping empty query strings.

- **Why It Was Rejected:** Vercel's caching behavior is managed at the Edge Router layer. Customizing query string normalization is restricted on standard plans. Disabling normalization increases cache key fragmentation (making cache hits less likely if query parameters are out of order), which degrades overall production application performance.

5. Code Diffs of the Applied Fixes

5.1 Client-Side Hook: `useCachedData.ts`

```
diff --git a/src/hooks/ad-performance/useCachedData.ts b/src/hooks/ad-performance/useCachedData.ts
```

```
@@ -321,6 +321,35 @@ export function useCachedData(): { const filters = { minSpend, minRoas, selectedChannels, creatorIds, productIds, editorIds, externalSources, statusFilter, search, showUnmappedAds }; const cacheKey = createCacheKey(fromDate, toDate, pageSize, sortBy, sortDirection, filters); + // Explicitly handle "deselect all channels" on the client so UI semantics are stable + // even if an environment treats `channels=` as "no filter". + if (selectedChannels !== null && selectedChannels.length === 0) { + dateRangeCache.delete(cacheKey); + setCurrentDateRange({ fromDate, toDate }); + setCurrentSort({ sortBy, sortDirection }); + setCurrentFilters(filters); + setCachedData([]); + setPaginationInfo({ + currentPage: page, + pageSize, + totalRecords: 0, + totalPages: 0, + hasNextPage: false, + hasPrevPage: false, + startIndex: 0, + endIndex: 0, + }); + setOverallTotals(null); + setMappedTotals(null); + setMappedTotalsExcludingPmax(null); + setUnmappedTotalsExcludingPmax(null); + setConceptLifecycle(null); + setChannelDistribution([]); + setLastFetched(new Date().toISOString()); + setLoading(false); + return []; + } + // For forced refreshes on page 1, drop any stale entry immediately so // later non-force reads cannot resurrect old data for the same key. if (page === 1 && forceRefresh) {
```

5.2 Client-Side Hook: `useComparisonConcepts.ts`

```
diff --git a/src/hooks/ad-performance/useComparisonConcepts.ts
b/src/hooks/ad-performance/useComparisonConcepts.ts
```

```
@@ -139,6 +139,15 @@ export function useComparisonConcepts({ return; } + if (debouncedChannels.length === 0) { +
```

```
setComparisonMap(EMPTY_COMPARISON_MAP); + setComparisonRange(null); + setComparisonOverallTotals(null); + setComparisonChannelDistribution(EMPTY_CHANNEL_DISTRIBUTION); + setComparisonLoading(false); + return; + } + const range = buildComparisonRange(); setComparisonRange((prev) => {
```

6. Functional Impact & Regression Analysis

We conducted a thorough regression review to ensure that bypassing the API request when no channels are selected does not break any other dashboard capabilities.

- **State Management Safety:** The short-circuit sets `loading` and `comparisonLoading` to `false` immediately. This prevents the UI from getting stuck in an infinite loading spinner.
- **Pagination Resets:** All pagination metadata (currentPage, totalPages, totalRecords) is reset to `0` or `false`. This prevents the UI from attempting to render empty page controls or throw index-out-of-bounds rendering errors.
- **Caching Integration:** When the user switches back from "None" to selecting specific channels, the standard URL-fetching path is restored. The module-level cache `dateRangeCache` works perfectly because the cache key continues to correctly hash the new channel parameters. No stale/empty states are cached under valid channel configurations.

7. QA Verification Playbook

To verify that the fix is stable, follow this manual QA plan.

Step 1: Localhost Verification

1. Open the dashboard locally.
2. Select multiple channels (e.g., Meta, TikTok, YouTube). Confirm data loads.
3. Click **"Deselect All"** (or deselect each channel manually).
4. Verify that the table instantly displays `0` concepts and all totals/distribution sections reset to zero. Confirm in Browser DevTools Network tab that no API request was made.

Step 2: Comparison Mode Verification

1. Toggle **"Compare Mode"** ON.
2. Deselect all channels.
3. Confirm that the comparison totals and comparison map reset to empty immediately and no

errors are thrown in the browser console.

Step 3: Production Validation (Vercel)

1. Navigate to the production app URL.
2. Repeat the channel deselect steps.
3. Confirm that production now behaves exactly like localhost: the table goes to empty immediately, no network requests are sent to the backend, and stale data does not persist.

8. Conclusion & Engineering Best Practices

- **Prefer Client-Side Interceptions for Zero-States:** When a user request logically evaluates to an empty result (e.g., searching for nothing, filtering by no categories, or selecting no options), short-circuit it client-side. This saves serverless resources, database execution costs, and provides an instant UI response.
- **Avoid Semantic Empty Query Parameters:** Never rely on query parameters with empty values (e.g., `?filter=`) to distinguish between "No filter" and "Filter by empty set" in server-side routing logic. If such a distinction is required, use explicit parameter keys or separate path structures.